Introduction to Strings

A **string** is a sequence of characters enclosed within single quotes (`' '`), double quotes (`" "`), or triple quotes (`''' '''` or `""" """`).
Strings are **immutable**, meaning once created, they cannot be changed.

Example:

```python
# Defining strings
str1 = 'Hello'
str2 = "World"
str3 = '''This is a multi-line
string example.'''

print(str1)
print(str2)
print(str3)
```

Output:

```none
Hello
World
This is a multi-line
string example.
```

Operations on Strings

1. **Concatenation (`+`)** - Joining two or more strings.
2. **Repetition (`*`)** - Repeating a string multiple times.
3. **Indexing** - Accessing individual characters using indices.
4. **Slicing** - Extracting a part (substring) of the string.
5. **Membership (`in`, `not in`)** - Checking if a substring exists within a string.

Examples:

```python
# Concatenation
str1 = "Hello"
str2 = "World"
result = str1 + " " + str2
print(result)  # Output: Hello World

# Repetition
print("Python! " * 3)  # Output: Python! Python! Python!

# Indexing
text = "Python"
print(text[0])    # Output: P (1st character)
print(text[-1])   # Output: n (last character)

# Slicing
print(text[0:3])  # Output: Pyt (from index 0 to 2)
print(text[:4])   # Output: Pyth (default start is 0)
print(text[2:])   # Output: thon (from index 2 to end)

# Membership
print('Pyt' in text)    # Output: True
print('Java' not in text)  # Output: True
```

Built-in String Functions

While string **methods** are called on string objects, Python also provides some **built-in functions** that work with strings:

1. `len()` - Returns the length of the string.
2. `max()` / `min()` - Returns the maximum or minimum character (based on ASCII values).
3. `sorted()` - Returns a sorted list of characters.
4. `chr()` / `ord()` - Converts between characters and their ASCII codes.

Examples:

```python
# 1. len()
text = "Hello World"
print(len(text))  # Output: 11

# 2. max() and min()
print(max("abcXYZ"))  # Output: 'c' (highest ASCII value)
print(min("abcXYZ"))  # Output: 'X' (lowest ASCII value)

# 3. sorted()
print(sorted("python"))  # Output: ['h', 'n', 'o', 'p', 't',
'y']

# 4. chr() and ord()
print(ord('A'))  # Output: 65 (ASCII value of 'A')
print(chr(97))   # Output: 'a' (character for ASCII value 97)
```

Complete List of Python String Methods with Examples

1. Case Conversion Methods

Method	Description	Example
`upper()`	Converts all characters to **uppercase**.	`"hello".upper()` → `"HELLO"`
`lower()`	Converts all characters to **lowercase**.	`"HELLO".lower()` → `"hello"`
`capitalize()`	Capitalizes the **first character**.	`"python".capitalize()` → `"Python"`
`title()`	Capitalizes the **first letter of each word**.	`"hello world".title()` → `"Hello World"`
`swapcase()`	**Swaps case**: upper → lower, lower → upper.	`"PyThOn".swapcase()` → `"pYtHoN"`
`casefold()`	Converts to lowercase (more aggressive than `lower()`).	`"STRAßEMÁLAGAÅngströmCafé".casefold()` → `"ss"`

2. Alignment & Formatting Methods

Method	Description	Example
`center(width, fillchar)`	Centers the string within `width`.	`"python".center(10, "*")` → `"*python*"`
`ljust(width, fillchar)`	Left-aligns the string within `width`.	`"py".ljust(5, "-")` → `"py---"`
`rjust(width, fillchar)`	Right-aligns the string within `width`.	`"py".rjust(5, "-")` → `"---py"`
`zfill(width)`	Pads the string with **zeros** on the left.	`"42".zfill(5)` → `"00042"`

3. Search & Find Methods

Method	Description	Example
`find(sub)`	Returns the **index** of first occurrence, `-1` if not found.	`"hello".find("l")` → 2
`rfind(sub)`	Returns the **last occurrence** of the substring.	`"hello".rfind("l")` → 3
`index(sub)`	Like `find()`, but raises an **error** if not found.	`"hello".index("e")` → 1
`rindex(sub)`	Like `rfind()`, but raises an **error** if not found.	`"hello".rindex("l")` → 3
`count(sub)`	Counts how many times `sub` appears.	`"banana".count("a")` → 3

4. String Testing Methods (Boolean Results)

Method	Description	Example
`startswith(sub)`	Checks if the string starts with `sub`.	`"python".startswith("py")` → `True`
`endswith(sub)`	Checks if the string ends with `sub`.	`"python".endswith("on")` → `True`
`isalpha()`	Returns `True` if all characters are **alphabets**.	`"Hello".isalpha()` → `True`
`isalnum()`	Returns `True` if all characters are **alphanumeric**.	`"abc123".isalnum()` → `True`
`islower()`	Returns `True` if all characters are **lowercase**.	`"hello".islower()` → `True`
`isupper()`	Returns `True` if all characters are **uppercase**.	`"HELLO".isupper()` → `True`
`isspace()`	Returns `True` if all characters are **whitespace**.	`" ".isspace()` → `True`
`istitle()`	Returns `True` if the string is in the title **case**.	`"Hello World".istitle()` → `True`
`isidentifier()`	Checks if the string is a valid **Python identifier**.	`"variable1".isidentifier()` → `True`

- `isdecimal()` → Most strict; only base-10 digits (`'0'–'9'` and equivalents)
- `isdigit()` → Allows additional digit characters like superscripts
- `isnumeric()` → Most flexible; includes digits, fractions, Roman numerals, etc.

- '5', # Standard decimal digit
- '٣', # Arabic-Indic digit (3)
- '۵', # Persian digit (5)
- '½', # Unicode fraction one-half
- '⅓', # Unicode fraction one-third

- '⅔', # Unicode fraction two-thirds
- '1/2', # Text-style fraction
- '2/3', # Text-style fraction
- 'VIII', # Roman numeral 8
- 'iv', # Roman numeral 4 (lowercase)
- '²', # Superscript 2
- '⓫', # Enclosed number 11 (not numeric)
- '١٢٣', # Arabic-Indic number (123)
- 'Ⅷ', # Roman numeral 8 (Unicode)
- '10', # Decimal number
- '0.5', # Float (with dot)
- '⅕', # Unicode fraction one-fifth

String	`isdecimal()`	`isdigit()`	`isnumeric()`	Description
`'123'`	True	True	True	ASCII decimal digits
`'٣'`	True	True	True	Arabic-Indic digit
`'²'`	False	True	True	Superscript 2
`'⅓'`	False	False	True	Fraction one-third
`'五'`	False	False	True	Chinese numeral for five
`'Ⅷ'`	False	False	True	Roman numeral 8
`'abc'`	False	False	False	Alphabetic characters
`'12.3'`	False	False	False	Contains a dot, not a numeric char
`'1a3'`	False	False	False	Contains non-numeric character

5. Replace & Modify Methods

Method	Description	Example
`replace(old, new)`	Replaces occurrences of `old` with `new`.	`"apple".replace("p", "b")` → `"abble"`
`translate(table)`	Replaces characters using a **translation table**.	`"abc".translate(str.maketrans("a", "x"))` → `"xbc"`
`maketrans()`	Creates a translation table for translate().	`"python".maketrans("aon", "%#5")` → {97: 37, 111: 35, 110: 53}

6. Splitting & Joining Methods

Method	Description	Example
`split(sep)`	Splits the string into a list by `sep`.	`"a,b,c".split(",")` → `['a', 'b', 'c']`
`rsplit(sep)`	Splits from the **right** side.	`"a,b,c".rsplit(",", 1)` → `['a,b', 'c']`
`splitlines()`	Splits at **line breaks** (`\n`).	`"Hello\nWorld".splitlines()` → `['Hello', 'World']`
`join(iterable)`	Joins elements with a **separator**.	`" ".join(["Hello", "World"])` → `"Hello World"`
`partition(sep)`	Splits into a **3-part tuple** at first `sep`.	`"apple-pie".partition("-")` → `('apple', '-', 'pie')`
`rpartition(sep)`	Splits into a **3-part tuple** at last `sep`.	`"apple-pie".rpartition("-")` → `('apple', '-', 'pie')`

7. Whitespace & Trimming Methods

Method	Description	Example
`strip(chars)`	Removes **leading and trailing** characters (default: spaces).	`" hello ".strip()` → `"hello"`
`lstrip(chars)`	Removes **leading** characters.	`"---hello".lstrip("-")` → `"hello"`
`rstrip(chars)`	Removes **trailing** characters.	`"hello---".rstrip("-")` → `"hello"`

8. Encoding & Decoding Methods

Method	Description	Example
`encode(encoding)`	Converts the string to **bytes**.	`"hello".encode("utf-8")` → `b'hello'`
`decode(encoding)`	Converts bytes back to **string**.	`b'hello'.decode("utf-8")` → `"hello"`

```
>>>text = "Hello नमस्ते 你好 café 🙂"
>>>encoded = text.encode("utf-8")
>>>print(encoded) #b'Hello
\xe0\xa4\xa8\xe0\xa4\xae\xe0\xa4\xb8\xe0\xa5\x8d\xe0\xa4\xa4\xe0\xa5\x87
\xe4\xbd\xa0\xe5\xa5\xbd caf\xc3\xa9 \xf0\x9f\x99\x82'
>>>b'Hello \xe0\xa4\xa8\xe0\xa4\xae\xe0\xa4\xb8\xe0\xa5\x8d\xe0\xa4\xa4\xe0\xa5\x87
\xe4\xbd\xa0\xe5\xa5\xbd caf\xc3\xa9 \xf0\x9f\x99\x82'.decode('utf-8')
>>>'Hello नमस्ते 你好 café 🙂'
```